Lecture Notes: TwinCAT 3

ADS (Automation Device Specification)

March 14, 2026

Contents

1 Introduction to ADS **2**
- 1.1 Internal Uses . . . 2
- 1.2 External Uses . . . 2
- 1.3 Protocol Details . . . 2

2 Addressing: AmsNetId and Ports **2**
- 2.1 AmsNetId . . . 2
- 2.2 ADS Ports . . . 3
- 2.3 ADS Message Router . . . 3

3 ADS Packet Structure **3**
- 3.1 Packet Layers . . . 3
- 3.2 Index Groups for PLC Access . . . 3

4 Practical Implementation: .NET / C# (Windows) **3**
- 4.1 Library Integration . . . 4
- 4.2 Symbolic Access . . . 4

5 Practical Implementation: C++ (Linux) **7**
- 5.1 Compilation . . . 7
- 5.2 Reverse Routing . . . 7
- 5.3 C++ API . . . 7

6 Summary of Benefits **11**

Quick Reference Card **11**

1 Introduction to ADS

Automation Device Specification (ADS) is Beckhoff's standardized interface for communication between various software modules within the TwinCAT environment. It utilizes a **client-server architecture**, where a connection is always initiated by a client to a server.

1.1 Internal Uses

ADS facilitates communication between the development environment (XAE) and the runtime (XAR) for reading/writing variables, and it is used internally by the TwinCAT scheduler to coordinate different modules.

1.2 External Uses

It allows external devices, such as a Raspberry Pi, Linux machines, or other TwinCAT runtimes, to exchange data with a target PLC.

1.3 Protocol Details

ADS can be **cyclic** (polling) or **event-based** (subscriptions). It typically uses **TCP ports 8016 and 48898** for communication and **UDP port 48899** for broadcasting device searches.

Port	Protocol	Purpose
48898	TCP	ADS communication
8016	TCP	Secure ADS (TLS)
48899	UDP	Device discovery broadcast

Table 1: ADS Network Ports

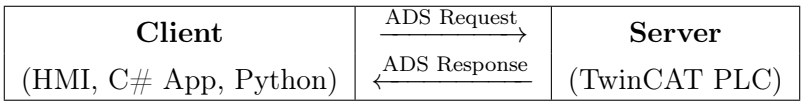

Figure 1: ADS Client-Server Architecture

2 Addressing: AmsNetId and Ports

To communicate via ADS, you must identify both the host device and the specific service running on that device.

2.1 AmsNetId

This is the unique address of a computer within the TwinCAT network. It consists of **six octets** (e.g., `192.168.4.200.1.1`). While often derived from the system's IP address by default, **the AmsNetId is not an IP address** and can be changed independently.

> **Note:** The AmsNetId format `x.x.x.x.1.1` is common, where the first four octets match the IP address and the last two are typically `1.1`. However, this is just a convention—the AmsNetId can be any six-octet value and does not need to correlate with the IP address.

2.2 ADS Ports

Within a host, different services are assigned specific ports.

Port	Service	Description
10000	System Service	Logger, Router
300	Real-time I/O	EtherCAT Master/Slaves
301	NC	Numerical Control
350	NC Interpolation	Path interpolation
500	CNC	Computer Numerical Control
801–804	Reserved	Legacy ports
851	PLC Runtime 1	First PLC instance
852	PLC Runtime 2	Second PLC instance
853	PLC Runtime 3	Third PLC instance
854	PLC Runtime 4	Fourth PLC instance

Table 2: Common ADS Port Assignments

2.3 ADS Message Router

This component defines communication channels (routes) between different TwinCAT devices. On Windows, you must manually add a route to a remote PLC to establish communication.

⚠ **Warning:** Before an external client can communicate with a TwinCAT PLC, a **route must be established on both sides**. The Windows TwinCAT system uses the AMS Router GUI, while Linux systems require manual configuration or reverse routing from the PLC side.

3 ADS Packet Structure

An AMS packet consists of several layers:

3.1 Packet Layers

- **AMS TCP Header:** Contains the length of the subsequent data.
- **AMS Header:** Contains the **Target/Source AmsNetId**, **Target/Source Port**, State Flags (request vs. response), and the **Command ID** (e.g., Read, Write).
- **ADS Data:** The payload, which varies by command. For a "Read" command, this includes the **Index Group**, **Index Offset**, and length of data.

3.2 Index Groups for PLC Access

4 Practical Implementation: .NET / C# (Windows)

For Windows-based applications, Beckhoff provides official libraries, including support for **.NET 6**, which offers cross-platform compatibility.

Command ID	Name	Description
0x0001	Read Device Info	Get device name and version
0x0002	Read	Read data from server
0x0003	Write	Write data to server
0x0004	Read State	Get ADS/device state
0x0005	Write Control	Change ADS state
0x0006	Add Notification	Subscribe to variable changes
0x0007	Delete Notification	Unsubscribe from notifications
0x0008	Device Notification	Notification data from server
0x0009	Read Write	Combined read and write

Table 3: ADS Command IDs

Index Group	Hex	Description
16384	0x4000	PLC Memory Area (%M)
16416	0x4020	PLC Data Area (variables)
16432	0x4030	Retain Data
61443	0xF003	Read by symbol name
61449	0xF009	Get symbol handle
61450	0xF00A	Release symbol handle
61451	0xF00B	Read by handle
61452	0xF00C	Write by handle

Table 4: Common Index Groups for PLC Access

4.1 Library Integration

The easiest method is using the **NuGet package manager** to install `Beckhoff.TwinCAT.ADS`.

4.2 Symbolic Access

While you can read data via Index Groups (e.g., **4020** for PLC memory), it is recommended to use **symbolic paths** (e.g., `MAIN.nCounter`). This ensures the code remains functional even if variable memory addresses change after recompilation.

Example: C# Symbolic Read

```
// C# Console Application: Reading a PLC Variable via ADS
// Required NuGet Package: Beckhoff.TwinCAT.Ads

using System;
using TwinCAT.Ads;

namespace AdsReadExample
{
    class Program
    {
        static void Main(string[] args)
        {
            // Create an ADS client instance
            // This is the main object for ADS communication
            using (AdsClient adsClient = new AdsClient())
            {
```

```
                try
                {
                    // Connect to the local TwinCAT runtime on port 851
                    // AmsNetId.Local represents the local machine (127.0.0.1.1.1)
                    // Port 851 is the default port for the first PLC runtime
                    adsClient.Connect(AmsNetId.Local, AmsPort.PlcRuntime_851);

                    Console.WriteLine("Connected to TwinCAT PLC on port 851");

                    // Read a variable using its symbolic path
                    // This is the RECOMMENDED approach as it survives recompilation
                    // The symbol path corresponds to: PROGRAM MAIN, variable nIntToRead
                    short valueRead = (short)adsClient.ReadValue(
                        "MAIN.nIntToRead",  // Symbolic variable path
                        typeof(short)        // Expected data type (INT = short in C#)
                    );

                    Console.WriteLine($"Value of MAIN.nIntToRead: {valueRead}");

                    // Alternative: Read using generic method for type safety
                    // This approach avoids casting and is more robust
                    int nCounter = adsClient.ReadValue<int>("MAIN.nCounter");
                    Console.WriteLine($"Value of MAIN.nCounter: {nCounter}");

                    // Read a REAL (float) value
                    float fTemperature = adsClient.ReadValue<float>("MAIN.fTemperature");
                    Console.WriteLine($"Temperature: {fTemperature:F2} C");

                    // Read a BOOL value
                    bool bMotorRunning = adsClient.ReadValue<bool>("MAIN.bMotorRunning");
                    Console.WriteLine($"Motor Running: {bMotorRunning}");

                    // Read a STRING value (note: TwinCAT strings have fixed length)
                    string sMessage = adsClient.ReadValue<string>("MAIN.sStatusMessage");
                    Console.WriteLine($"Status: {sMessage}");
                }
                catch (AdsException ex)
                {
                    // Handle ADS-specific errors
```

```
                    Console.WriteLine($"ADS Error: {ex.Message}");
                    Console.WriteLine($"Error Code: {ex.ErrorCode}");
                }
                catch (Exception ex)
                {
                    // Handle general errors
                    Console.WriteLine($"Error: {ex.Message}");
                }
                finally
                {
                    // Disconnect is handled by 'using' statement
                    Console.WriteLine("Disconnected from PLC");
                }
            }

            Console.WriteLine("Press any key to exit...");
            Console.ReadKey();
        }
    }
}
```

```
// Extended Example: Reading from a Remote PLC

using System;
using TwinCAT.Ads;

namespace AdsRemoteReadExample
{
    class Program
    {
        static void Main(string[] args)
        {
            using (AdsClient adsClient = new AdsClient())
            {
                // For remote PLC, specify the AmsNetId explicitly
                // This requires a route to be configured in TwinCAT Router
                string remoteAmsNetId = "192.168.1.100.1.1";

                // Parse the AmsNetId string to an AmsNetId object
                AmsNetId targetNetId = AmsNetId.Parse(remoteAmsNetId);

                // Connect to remote PLC runtime 1 (port 851)
                adsClient.Connect(targetNetId, AmsPort.PlcRuntime_851);

                Console.WriteLine($"Connected to remote PLC: {remoteAmsNetId}");

                // Read values from the remote PLC
                int nValue = adsClient.ReadValue<int>("MAIN.nIntToRead");
                Console.WriteLine($"Remote value: {nValue}");

                // Write a value to the remote PLC
                adsClient.WriteValue("MAIN.nIntToWrite", 12345);
                Console.WriteLine("Wrote value 12345 to MAIN.
```

```
    nIntToWrite");
            }
        }
    }
}
```

Tip: Always use symbolic access (`ReadValue<T>("MAIN.nVariable")`) instead of raw Index Group/Offset access. Symbolic access automatically resolves variable addresses at runtime, so your application continues to work even after the PLC program is recompiled and variable addresses change.

5 Practical Implementation: C++ (Linux)

Beckhoff provides an **open-source ADS client library** on GitHub for non-Windows environments.

5.1 Compilation

On Linux (e.g., Debian), the library is compiled from source using tools like **Meson and Ninja**.

```
# Clone the repository
git clone https://github.com/Beckhoff/ADS.git
cd ADS

# Build using Meson and Ninja
meson setup build
ninja -C build

# Install (optional)
sudo ninja -C build install
```

5.2 Reverse Routing

Since Linux lacks the standard TwinCAT AMS Router GUI, you must **authorize the connection on the PLC side**. This involves adding a route on the target PLC pointing back to the Linux client's IP address and its generated AmsNetId.

Note: On Linux, the ADS library generates an AmsNetId based on the system's hostname. You can find this ID in the library output or specify a custom one. The Windows TwinCAT PLC must have a route configured to this AmsNetId for communication to work.

5.3 C++ API

The C++ library allows for high-performance reading and writing. It supports overloading assignment operators, making the code very concise.

Example: C++ Write Operation

```cpp
// C++ ADS Client Example: Writing to a TwinCAT PLC Variable
// Requires: Beckhoff ADS open-source library from GitHub
// https://github.com/Beckhoff/ADS

#include <iostream>
#include <string>
#include "AdsLib.h"
#include "AdsVariable.h"

int main()
{
    // Define the target PLC's AmsNetId (six octets)
    // This must match the AmsNetId configured on the TwinCAT system
    static const AmsNetId remoteNetId { 192, 168, 1, 100, 1, 1 };

    // Define the target PLC's IP address
    // This is the actual network IP, separate from the AmsNetId
    static const char remoteIpAddress[] = "192.168.1.100";

    try
    {
        // Create an ADS device connection
        // Parameters: IP Address, AmsNetId, ADS Port (851 for PLC Runtime 1)
        AdsDevice adsDevice {
            remoteIpAddress,
            remoteNetId,
            AMSPORT_R0_PLC_TC3  // Port 851
        };

        std::cout << "Connected to PLC at " << remoteIpAddress << std::endl;

        // Create an ADS variable handle for writing
        // Template parameter specifies the data type (int32_t = DINT)
        // The string parameter is the symbolic path in the PLC
        AdsVariable<int32_t> nIntToWrite {
            adsDevice,
            "MAIN.nIntToWrite"
        };

        // Write a value to the PLC variable
        // The library overloads the assignment operator for clean syntax
        // This single line performs the entire ADS write operation
        nIntToWrite = 42;

        std::cout << "Successfully wrote value 42 to MAIN.nIntToWrite" << std::endl;

        // Create another variable for reading back
        AdsVariable<int32_t> nIntToRead {
            adsDevice,
            "MAIN.nIntToRead"
        };

```

```
        // Read the value - implicit conversion via overloaded operator
        int32_t readValue = nIntToRead;
        std::cout << "Read value from MAIN.nIntToRead: " << readValue << std::endl;

        // Example with different data types

        // REAL (float)
        AdsVariable<float> fTemperature { adsDevice, "MAIN.fTemperature" };
        float tempValue = fTemperature;
        std::cout << "Temperature: " << tempValue << " C" << std::endl;

        // BOOL
        AdsVariable<bool> bMotorRunning { adsDevice, "MAIN.bMotorRunning" };
        bMotorRunning = true;  // Start the motor
        std::cout << "Motor started" << std::endl;

        // Writing multiple values in sequence
        AdsVariable<int32_t> nSetpoint { adsDevice, "MAIN.nSetpoint" };
        for (int i = 0; i <= 100; i += 10)
        {
            nSetpoint = i;
            std::cout << "Setpoint: " << i << "%" << std::endl;
            // In real application, add delay here
        }

    }
    catch (const AdsException& ex)
    {
        // Handle ADS-specific errors
        std::cerr << "ADS Error: " << ex.what() << std::endl;
        std::cerr << "Error Code: " << ex.errorCode << std::endl;
        return 1;
    }
    catch (const std::exception& ex)
    {
        // Handle general errors
        std::cerr << "Error: " << ex.what() << std::endl;
        return 1;
    }

    std::cout << "Program completed successfully" << std::endl;
    return 0;
}
```

```
// Alternative C++ Example: Using Low-Level API
// For cases where you need more control over the ADS operations

#include <iostream>
#include "AdsLib.h"

int main()
{
```

```cpp
    // Add a route to the remote PLC
    // This is necessary on Linux since there's no AMS Router GUI
    long nPort = AdsPortOpenEx();

    if (nPort == 0)
    {
        std::cerr << "Failed to open ADS port" << std::endl;
        return 1;
    }

    // Define remote address
    AmsAddr remoteAddr;
    remoteAddr.netId = { 192, 168, 1, 100, 1, 1 };
    remoteAddr.port = AMSPORT_R0_PLC_TC3;  // 851

    // Add route (IP address of remote PLC)
    AdsAddRoute(remoteAddr.netId, "192.168.1.100");

    // Write a DINT value using Index Group and Offset
    // Index Group 0x4020 = PLC data area
    // Index Offset = memory offset of variable (from symbol info)
    int32_t valueToWrite = 42;
    long nResult = AdsSyncWriteReqEx(
        nPort,
        &remoteAddr,
        0x4020,            // Index Group: PLC Memory
        0x0000,            // Index Offset: variable offset
        sizeof(valueToWrite),
        &valueToWrite
    );

    if (nResult != ADSERR_NOERR)
    {
        std::cerr << "Write failed with error: " << nResult << std::endl;
    }
    else
    {
        std::cout << "Write successful" << std::endl;
    }

    // Clean up
    AdsPortCloseEx(nPort);

    return 0;
}
```

Warning: When using the C++ ADS library on Linux, ensure that:

1. A route is added on the Windows TwinCAT PLC pointing to your Linux machine's IP and AmsNetId
2. The firewall allows TCP port 48898 and UDP port 48899
3. The AmsNetId used in your code matches what's configured in the route

6 Summary of Benefits

- **Open System:** ADS allows TwinCAT to communicate with almost any language (C#, C++, Java, Python, Node.js) and platform.
- **No Licensing Fees:** Using ADS client libraries is **free** and does not require additional TwinCAT licenses.
- **Full Access:** ADS provides access not just to PLC variables, but also to motion control, I/O, vision, and system analytics.

Platform	Library	Source	License
Windows/.NET	Beckhoff.TwinCAT.Ads	NuGet	Free
Linux/C++	ADS Library	GitHub	MIT
Python	pyads	PyPI	MIT
Node.js	ads-client	npm	MIT
Java	AdsToJava	GitHub	Free

Table 5: ADS Client Libraries by Platform

Tip: ADS provides a unified interface to access all TwinCAT subsystems. Beyond PLC variables, you can use ADS to:

- Read/write NC axis parameters (Port 500/501)
- Access I/O terminals directly (Port 300)
- Query system diagnostics and logs (Port 10000)
- Interface with TwinCAT Vision (Port 2400)

Quick Reference Card

ADS Addressing Format

```
AmsNetId: x.x.x.x.x.x (six octets, e.g., 192.168.1.100.1.1)
ADS Port: 851 (PLC Runtime 1), 852 (Runtime 2), etc.
Full Address: 192.168.1.100.1.1:851
```

Common ADS Ports

Port	Service
300	I/O (EtherCAT)
851–854	PLC Runtimes 1–4
500	NC/CNC
10000	System Service

C# Quick Reference

```csharp
// Connect
using AdsClient client = new AdsClient();
client.Connect(AmsNetId.Local, AmsPort.PlcRuntime_851);

// Read
int value = client.ReadValue<int>("MAIN.nVariable");

// Write
client.WriteValue("MAIN.nVariable", 42);
```

C++ Quick Reference

```cpp
// Connect
AdsDevice device { "192.168.1.100", {192,168,1,100,1,1}, 851 };

// Create variable handle
AdsVariable<int32_t> myVar { device, "MAIN.nVariable" };

// Read
int32_t value = myVar;

// Write
myVar = 42;
```

Last Updated: March 14, 2026